

Lecture 10 - Attention and Transformers

Ertunc Erdil

April 2026

Attention is the fundamental block of Transformers

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder–decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder–decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

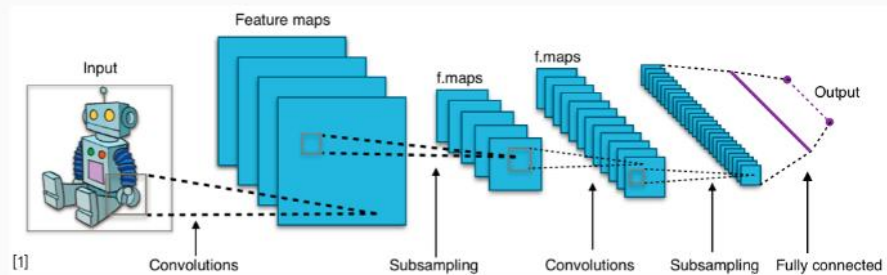
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

The classical landscape : One architecture per “community”

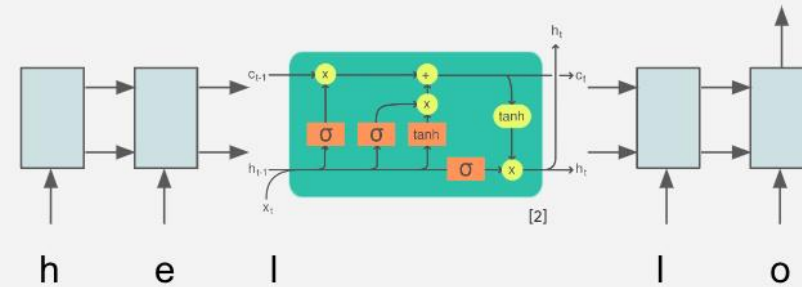
Computer Vision

Convolutional NNs (+ResNets)



Natural Lang. Proc.

Recurrent NNs (+LSTMs)



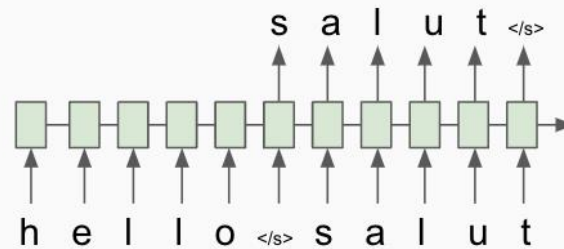
Speech

Deep Belief Nets (+non-DL)



Translation

Seq2Seq



RL

BC/GAIL

Algorithm 1 Generative adversarial imitation learning

- 1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
- 2: **for** $i = 0, 1, 2, \dots$ **do**
- 3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
- 4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i} [\nabla_w \log(D_w(s, a))] + \hat{\mathbb{E}}_{\tau_E} [\nabla_w \log(1 - D_w(s, a))] \quad (17)$$

- 5: Take a policy step from θ_i to θ_{i+1} , using the TRPO rule with cost function $\log(D_{w_{i+1}}(s, a))$. Specifically, take a KL-constrained natural gradient step with

$$\hat{\mathbb{E}}_{\tau_i} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q(s, a)] - \lambda \nabla_{\theta} H(\pi_{\theta}), \quad (18)$$

where $Q(\bar{s}, \bar{a}) = \hat{\mathbb{E}}_{\tau_i} [\log(D_{w_{i+1}}(s, a)) | s_0 = \bar{s}, a_0 = \bar{a}]$

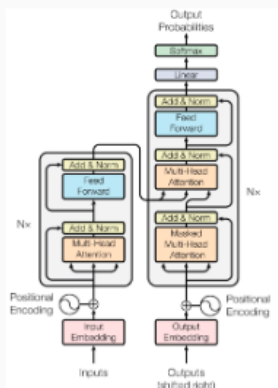
6: **end for**

[1] CNN image CC-BY-SA by Aphex34 for Wikipedia https://commons.wikimedia.org/wiki/File:Typical_cnn.png

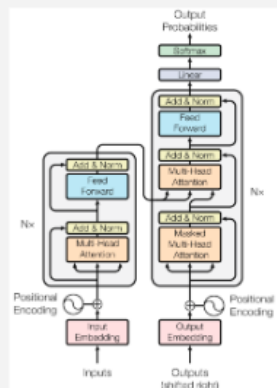
[2] RNN image CC-BY-SA by GChе for Wikipedia https://commons.wikimedia.org/wiki/File:The_LSTM_Cell.svg

The transformer’s takeover : One community at a time

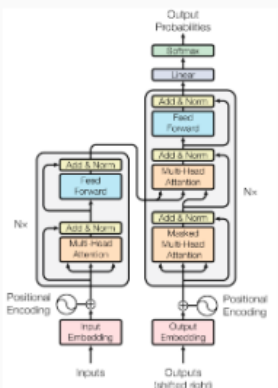
Computer Vision



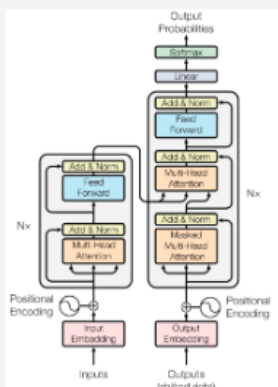
Natural Lang. Proc.



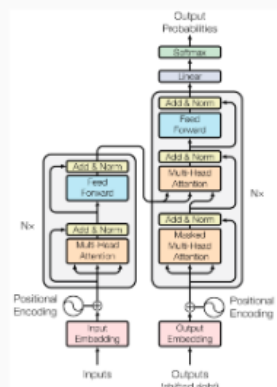
Reinf. Learning



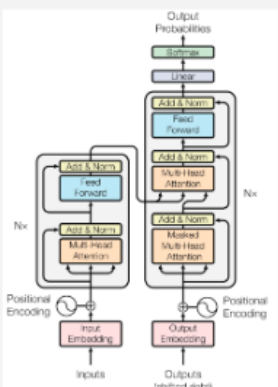
Speech



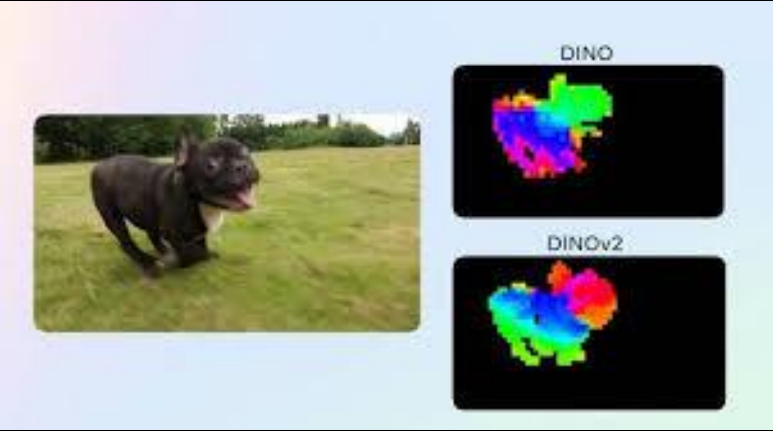
Translation



Graphs/Science



Transformers



Outline of this lecture

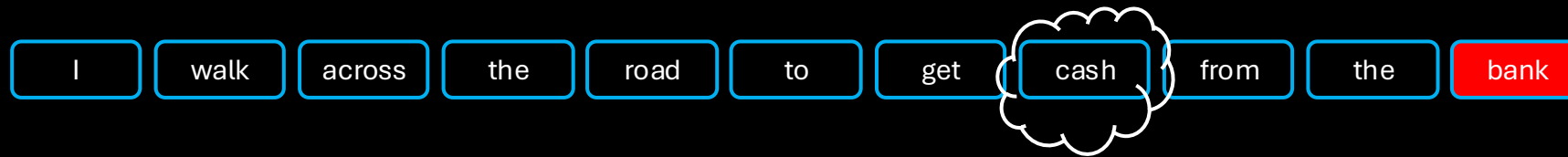
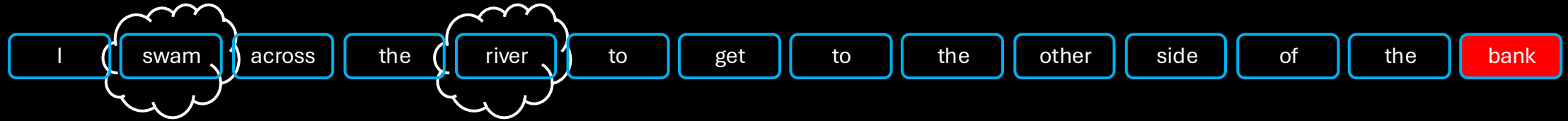
- Motivation
- Self-attention
- Scaled self-attention
- Multi-head self-attention
- Transformer layers
- Positional Encoding
- Transformers for images

Motivation

I swam across the river to get to the other side of the bank

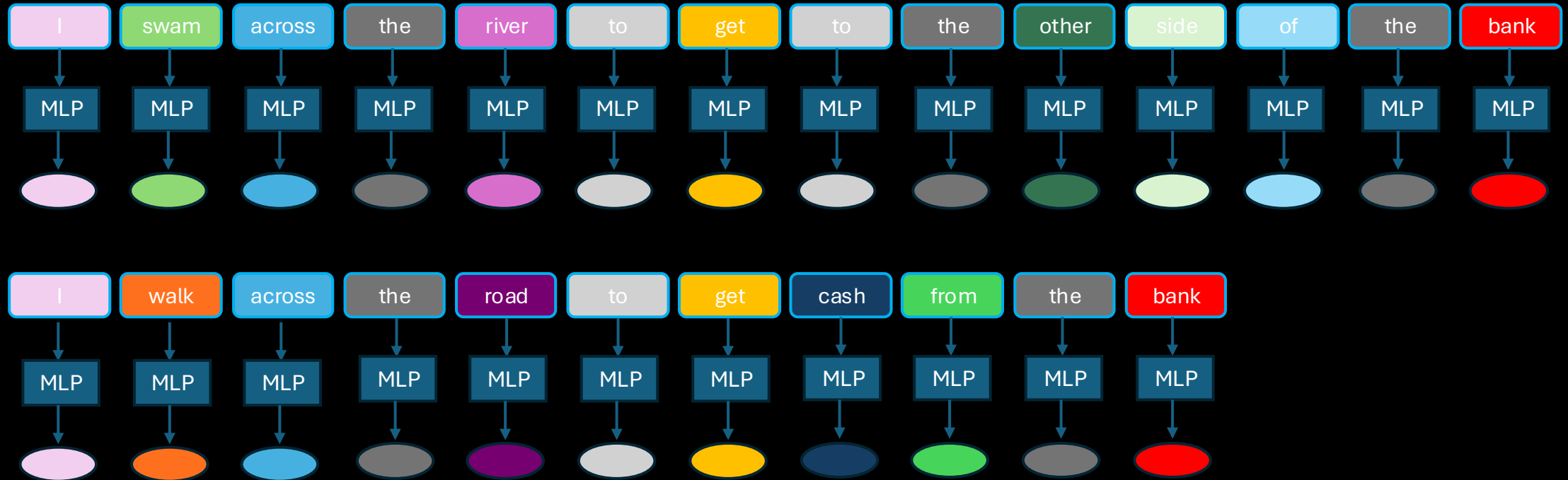
I walk across the road to get cash from the bank

Motivation



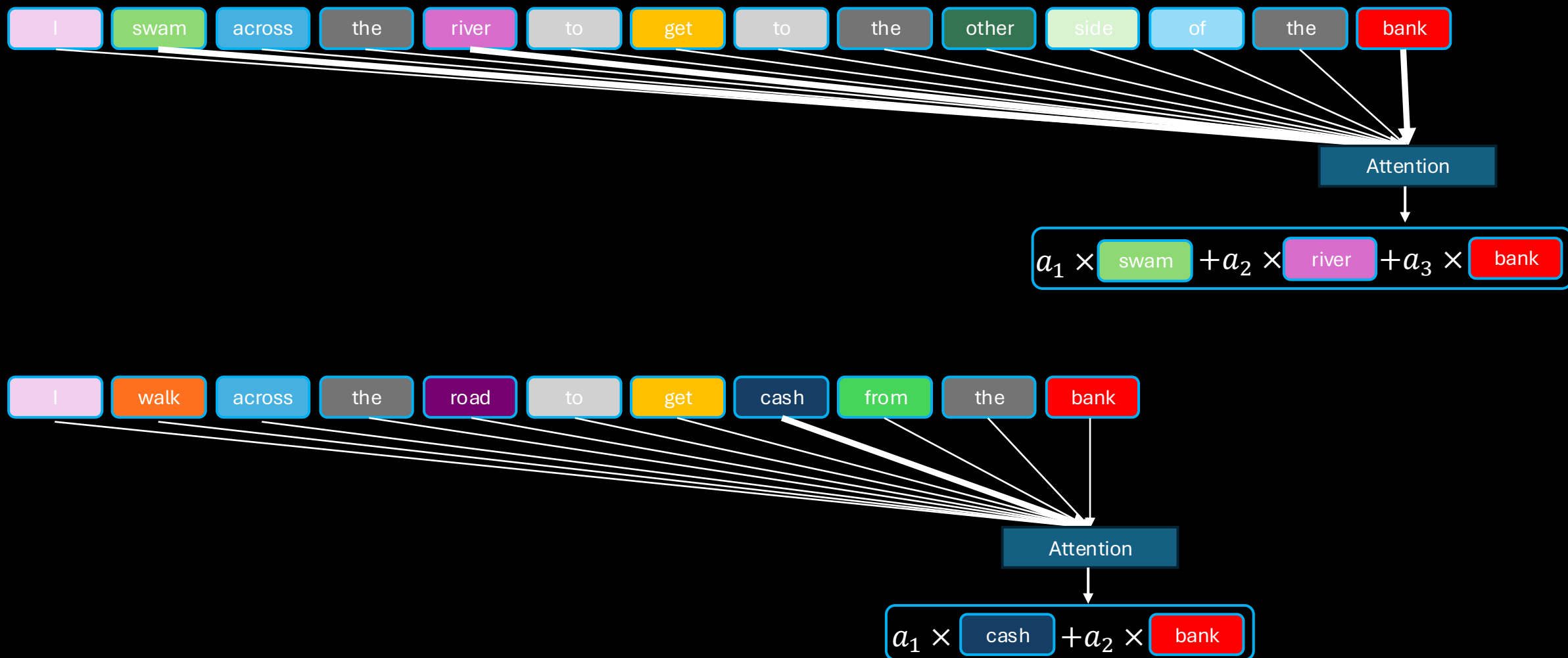
- Some words are more important than the others!

Motivation



- Standard network processing with, e.g., MLP $h = \sigma(Wx + b)$

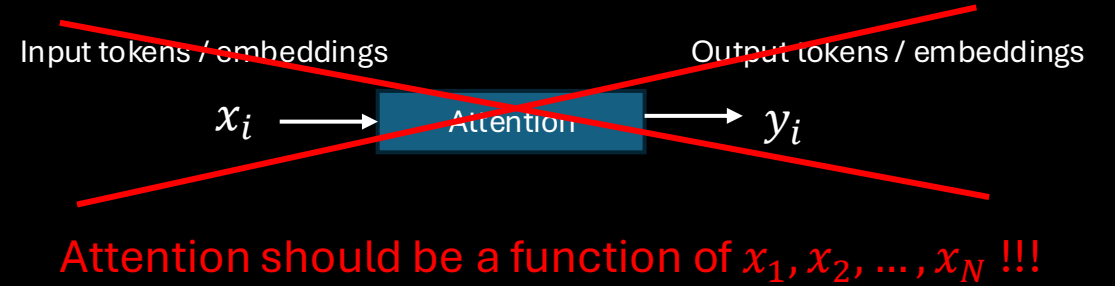
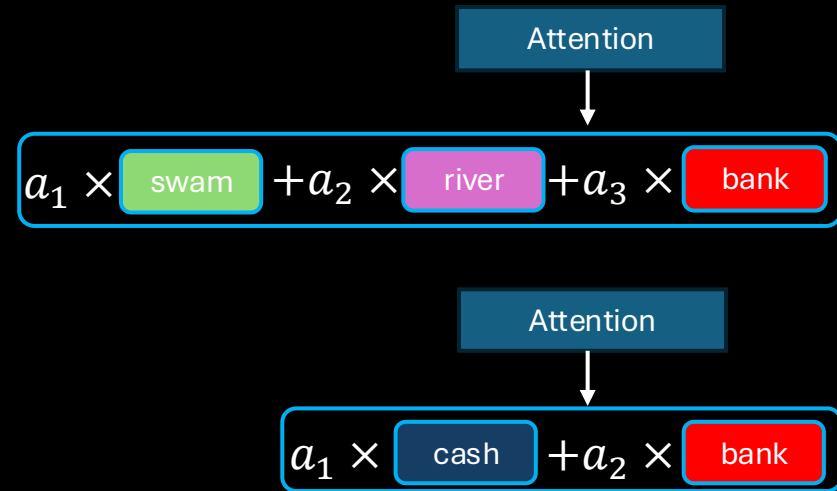
Motivation



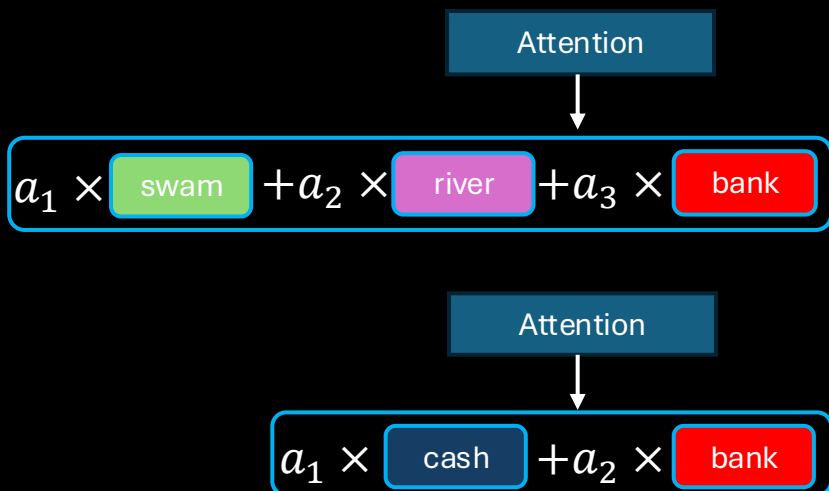
Outline of this lecture

- ✓ Motivation
- Self-attention
- Scaled self-attention
- Multi-head self-attention
- Transformer layers
- Positional Encoding
- Transformers for images

Self-attention



Self-attention



Attention should be a function of $x_1, x_2, \dots, x_N$!!!

$$y_i = \text{Attention}(x_1, \dots, \mathbf{x}_i, \dots, x_N) = \sum_{j=1}^N a_{ij} x_j$$

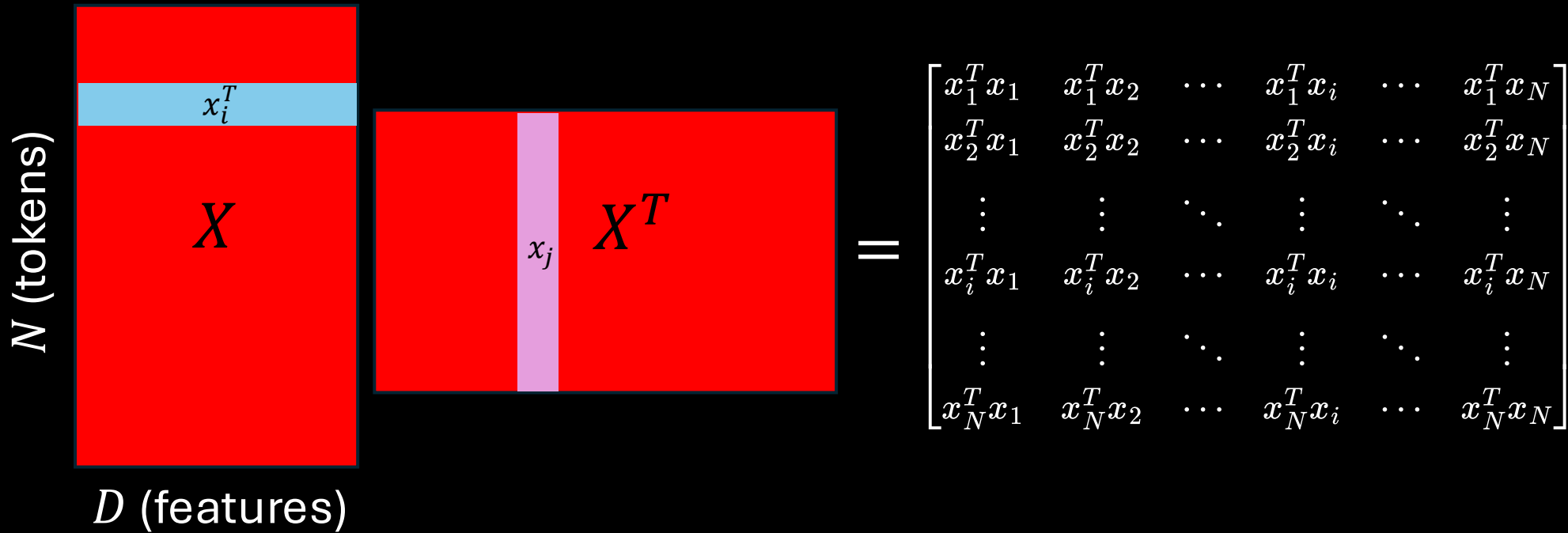
with constraints: $a_{ij} \geq 0$

$$\sum_{j=1}^N a_{ij} = 1$$

$$a_{ij} = \frac{\exp(x_i^T x_j)}{\sum_{j'=1}^N \exp(x_i^T x_{j'})}, x_i \in \mathbb{R}^{D \times 1}$$

Self-attention

- Matrix notation



Attention for a single token

$$y_i = \text{Attention}(x_1, \dots, \mathbf{x_i}, \dots, x_N) = \sum_{j=1}^N a_{ij} x_j$$

$$a_{ij} = \frac{\exp(x_i^T x_j)}{\sum_{j'=1}^N \exp(x_i^T x_{j'})}, x_i \in \mathbb{R}^{D \times 1}$$

Self-attention

- Matrix notation

Attention for a single token

$$y_i = \text{Attention}(x_1, \dots, \mathbf{x}_i, \dots, x_N) = \sum_{j=1}^N a_{ij} x_j$$

$$a_{ij} = \frac{\exp(x_i^T x_j)}{\sum_{j'=1}^N \exp(x_i^T x_{j'})}, x_i \in \mathbb{R}^{D \times 1}$$

$$\begin{bmatrix} x_1^T x_1 & x_1^T x_2 & \cdots & x_1^T x_i & \cdots & x_1^T x_N \\ x_2^T x_1 & x_2^T x_2 & \cdots & x_2^T x_i & \cdots & x_2^T x_N \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_i^T x_1 & x_i^T x_2 & \cdots & x_i^T x_i & \cdots & x_i^T x_N \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_N^T x_1 & x_N^T x_2 & \cdots & x_N^T x_i & \cdots & x_N^T x_N \end{bmatrix} \begin{matrix} \xrightarrow{\text{Softmax}} \\ \xrightarrow{\text{Softmax}} \\ \xrightarrow{\text{Softmax}} \\ \xrightarrow{\text{Softmax}} \end{matrix} \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1N} \\ a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2N} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{ij} & \cdots & a_{iN} \\ \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & \cdots & a_{Nj} & \cdots & a_{NN} \end{bmatrix}$$

!!! Softmax is applied to each row separately !!!

Self-attention

- Matrix notation

Attention for a single token

$$y_i = \text{Attention}(x_1, \dots, \mathbf{x}_i, \dots, x_N) = \sum_{j=1}^N a_{ij} x_j$$

$$a_{ij} = \frac{\exp(x_i^T x_j)}{\sum_{j'=1}^N \exp(x_i^T x_{j'})}, x_i \in \mathbb{R}^{D \times 1}$$

$$Y = \text{Softmax}(XX^T)X$$

!!! Softmax is applied to each row separately !!!

Self-attention

$$Y = \text{Softmax}(XX^T)X$$

- Any issue?
- No capacity to learn from data
- Each feature value within a token plays an equal role in determining the attention coefficients.

$$\begin{matrix} \overbrace{x_{i1}^T, x_{i2}^T, \dots, x_{iD}^T}^{x_i^T} & \begin{matrix} x_{j1} \\ x_{j2} \\ \vdots \\ x_{jD} \end{matrix} \end{matrix} \quad \left. \vphantom{\begin{matrix} x_{j1} \\ x_{j2} \\ \vdots \\ x_{jD} \end{matrix}} \right\} x_j^T$$

- Solution?
- Add learnable parameters

Self-attention

- Add learnable parameters

$$Y = \textit{Softmax}(XX^T)X$$

$$\tilde{X} = XU, U \in \mathbb{R}^{D \times D}$$

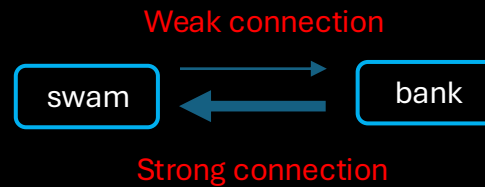
$$Y = \textit{Softmax}(\tilde{X}\tilde{X}^T)\tilde{X}$$

$$Y = \textit{Softmax}(XUU^TX^T)XU$$

Self-attention

$$Y = \text{Softmax}(XUU^TX^T)XU$$

- Any issue?
- ~~No capacity to learn from data.~~ Added learnable parameters.
- ~~Each feature value within a token plays an equal role in determining the attention coefficients.~~ Learnable parameters can help selecting more useful features.
- XUU^TX^T is symmetric.



- Solution?
- Add different learnable parameters for Key, Query, and Value

Self-attention

$$Y = \text{Softmax}(XUU^TX^T)XU$$

- Add different learnable parameters for Key, Query, and Value

$$K = XW_K, W_K \in \mathbb{R}^{D \times D_K}$$

$$Q = XW_Q, W_Q \in \mathbb{R}^{D \times D_Q}$$

$$V = XW_V, W_V \in \mathbb{R}^{D \times D_V}$$

$$Y = \text{Softmax}(QK^T)V$$

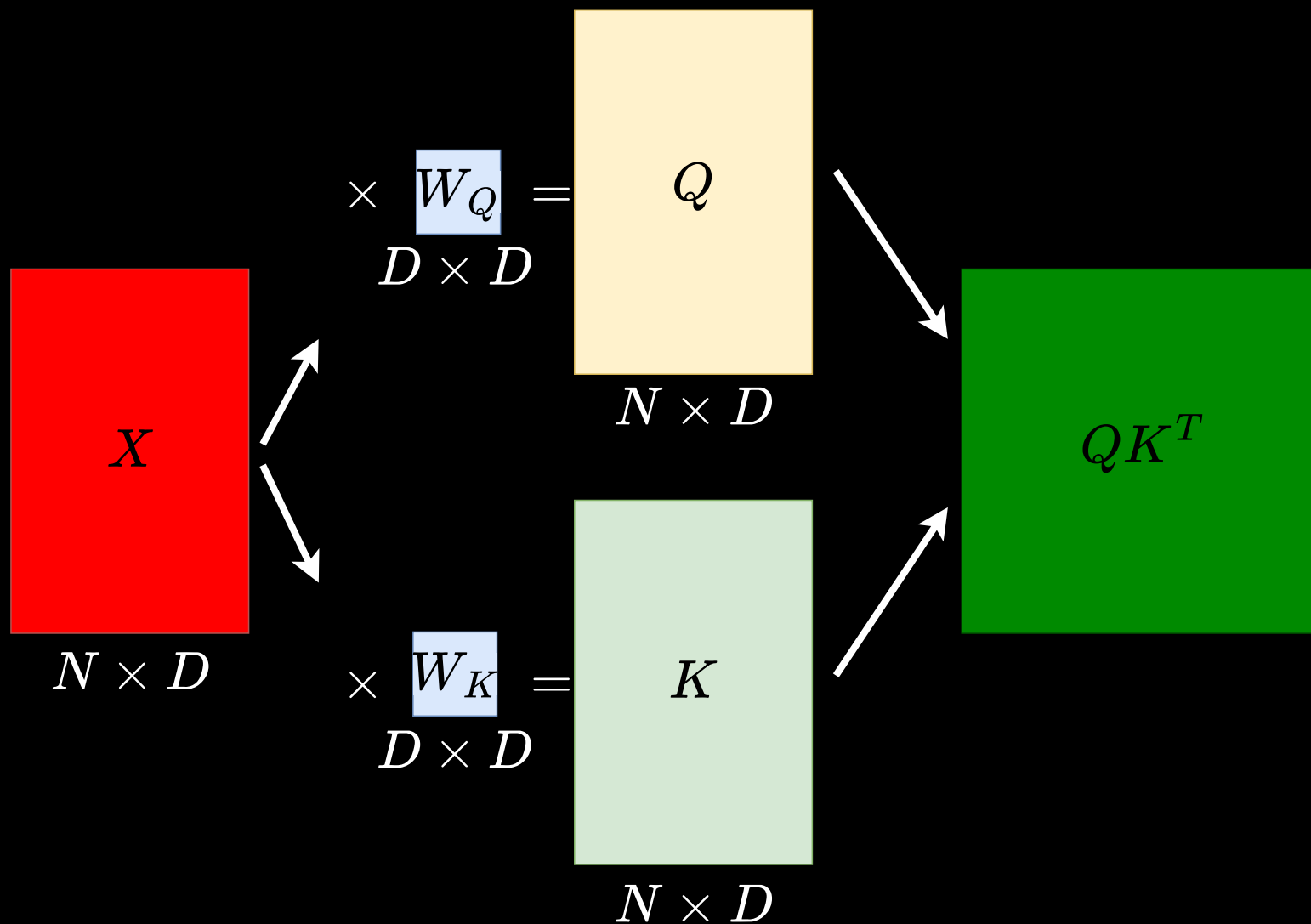
Self-attention

$$Y = \text{Softmax}(QK^T)V$$

- Any issue?
- ~~No capacity to learn from data.~~ Added learnable parameters.
- ~~Each feature value within a token plays an equal role in determining the attention coefficients.~~ Learnable parameters can help selecting more useful features.
- ~~XUU^TX^T is symmetric.~~ QK^T is asymmetric.

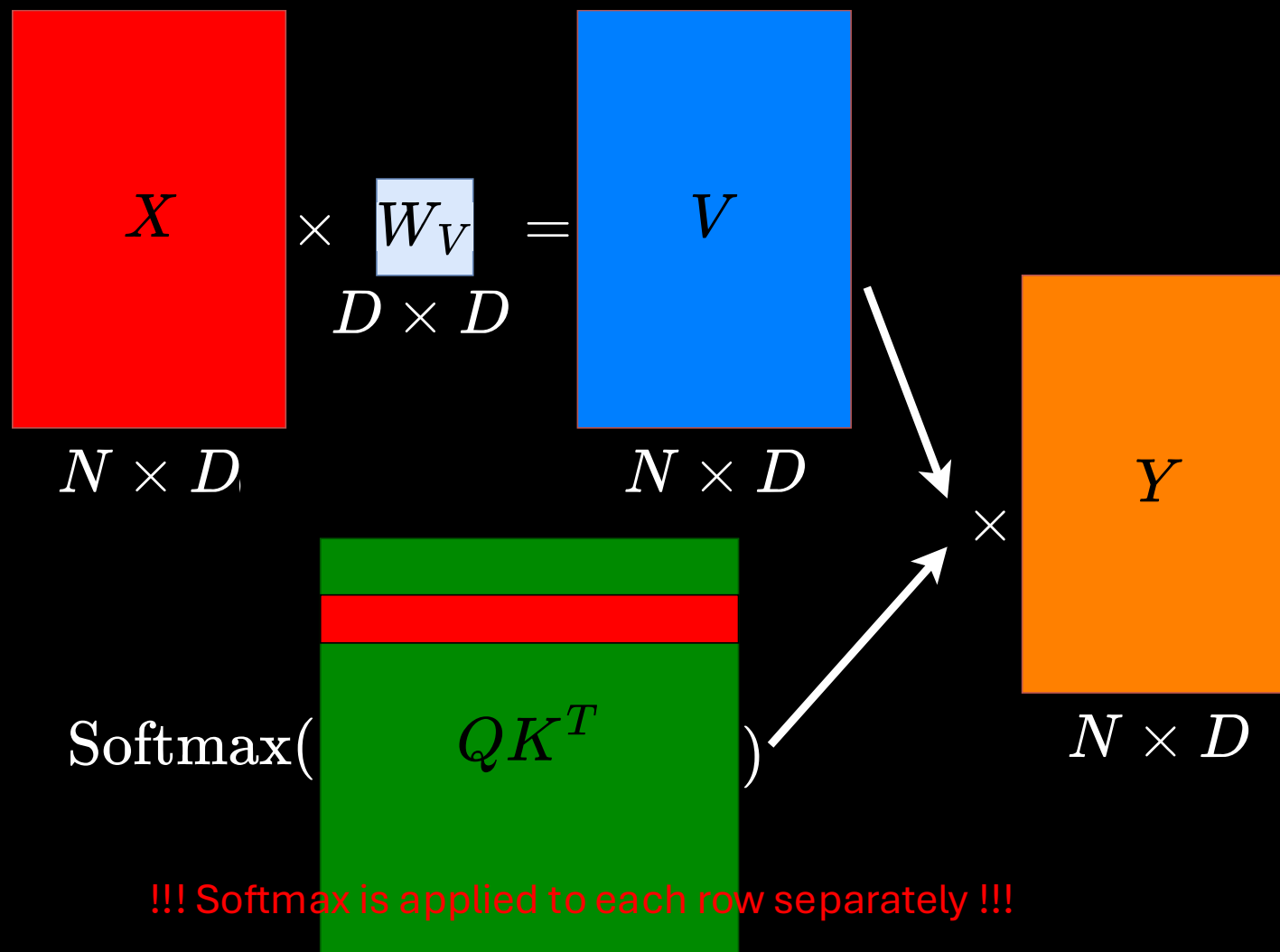
Self-attention

$$Y = \text{Softmax}(QK^T)V$$



Self-attention

$$Y = \text{Softmax}(QK^T)V$$



!!! Softmax is applied to each row separately !!!

Self-attention

$$Y = \text{Softmax}(QK^T)V$$

- Information retrieval terminology – Key, Query, Value



- Classic information retrieval: Hard attention – we select the most similar movie to the query
- In transformers:
 - Generalize hard attention to soft attention
 - Use continuous variables to measure the degree of match between query and keys

Outline of this lecture

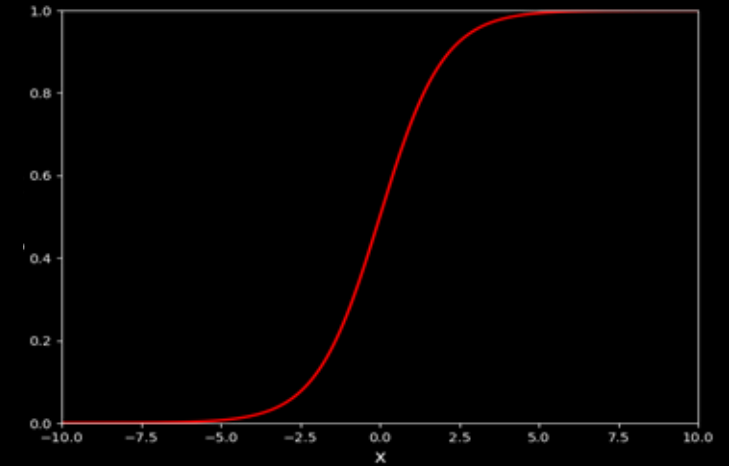
- ✓ Motivation
- ✓ Self-attention
 - Scaled self-attention
 - Multi-head self-attention
 - Transformer layers
 - Positional Encoding
 - Transformers for images

Scaled self-attention

- The gradients of the softmax function become exponentially small for small and large inputs.
- If $Q, K \sim N(0, I)$ then $\text{Var}(QK^T) = D_K$.

Self-attention

$$Y = \text{Softmax}(QK^T)V$$



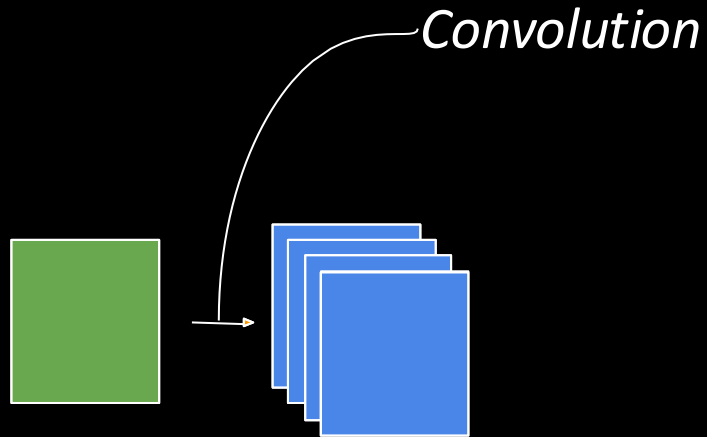
$$Y = \text{Softmax}\left(\frac{QK^T}{\sqrt{D_K}}\right)V$$

Outline of this lecture

- ✓ Motivation
- ✓ Self-attention
- ✓ Scaled self-attention
- Multi-head self-attention
- Transformer layers
- Positional Encoding
- Transformers for images

Multi-head self-attention

- Remember multiple channels in CNNs.



- Scaled 'single-head' self-attention

$$Y = \text{Softmax} \left(\frac{QK^T}{\sqrt{D_K}} \right) V$$

- There might be different useful features to extract at each layer.
- Use multiple filters.

Multi-head self-attention

- There might be multiple patterns that require attention.
- Use multiple attention heads.

$$H_h = \text{Attention}(Q_h, K_h, V_h), \forall h \in [1, H]$$

$$\text{Attention}(Q_h, K_h, V_h) = \text{Softmax}\left(\frac{Q_h K_h^T}{\sqrt{D_{K_h}}}\right) V_h$$

$$K_h = XW_{K_h}$$

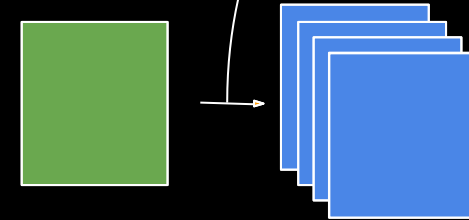
$$Q_h = XW_{Q_h}$$

$$V_h = XW_{V_h}$$

- Scaled 'single-head' self-attention

$$Y = \text{Softmax}\left(\frac{QK^T}{\sqrt{D_K}}\right) V$$

- Remember multiple channels in CNNs.



- There might be different useful features to extract at each layer.
- Use multiple filters.

Multi-head self-attention

- There might be multiple patterns that require attention.
- Use multiple attention heads.

$$H_h = \text{Attention}(Q_h, K_h, V_h), \forall h \in [1, H]$$

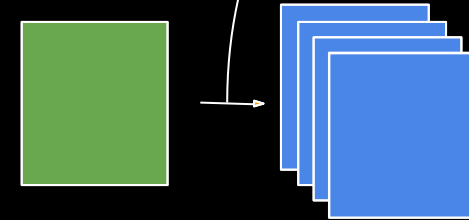
$$Y = \text{Concat}[H_1, H_2, \dots, H_H] W_0$$

$$\text{Concat}[H_1, H_2, \dots, H_H] \in \mathbb{R}^{N \times HD_V} \quad W_0 \in \mathbb{R}^{HD_V \times D}$$

- Scaled 'single-head' self-attention

$$Y = \text{Softmax}\left(\frac{QK^T}{\sqrt{D_K}}\right) V$$

- Remember multiple channels in CNNs.

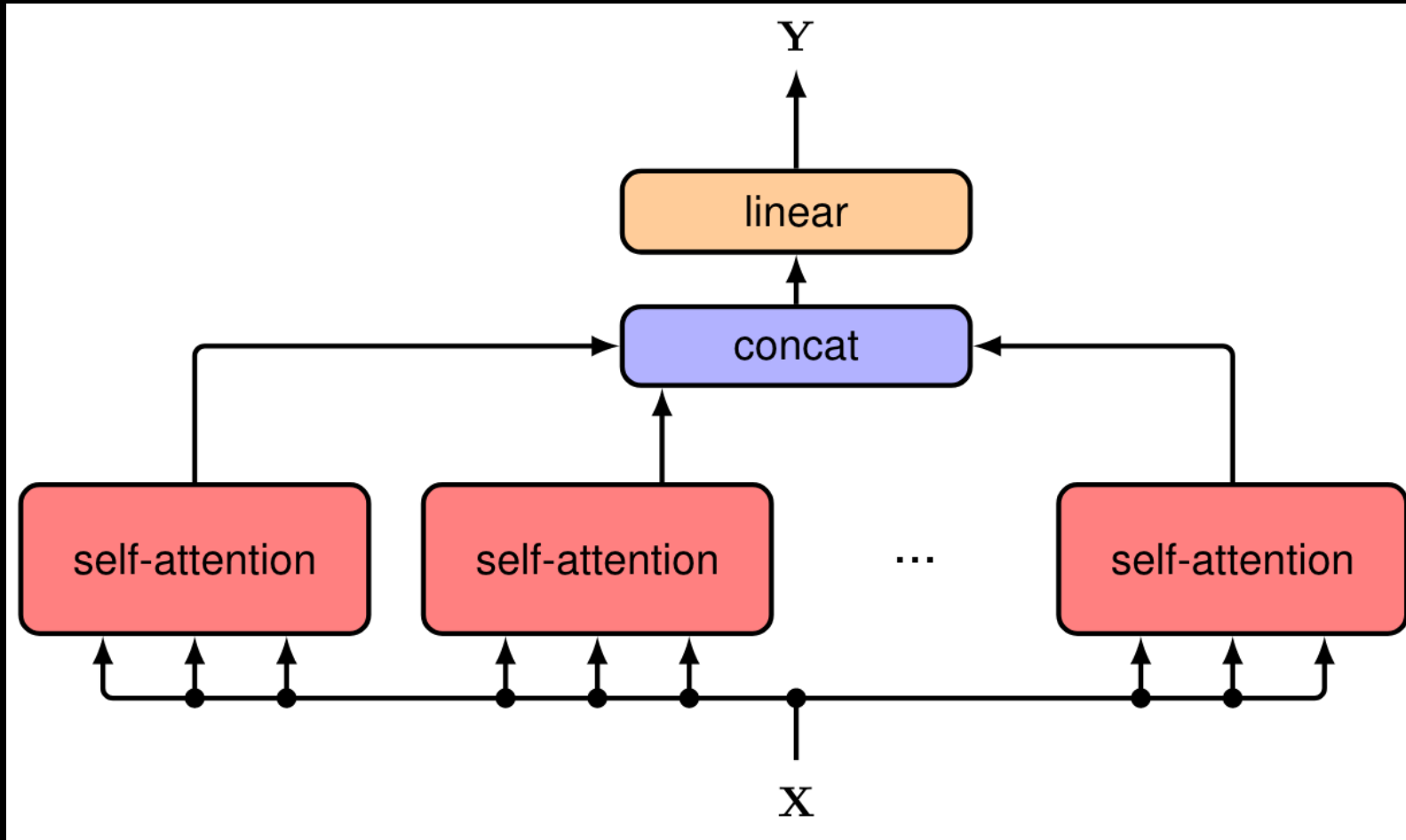


- There might be different useful features to extract at each layer.
- Use multiple filters.

Multi-head self-attention

$$H_h = \text{Attention}(Q_h, K_h, V_h), \forall h \in [1, H]$$

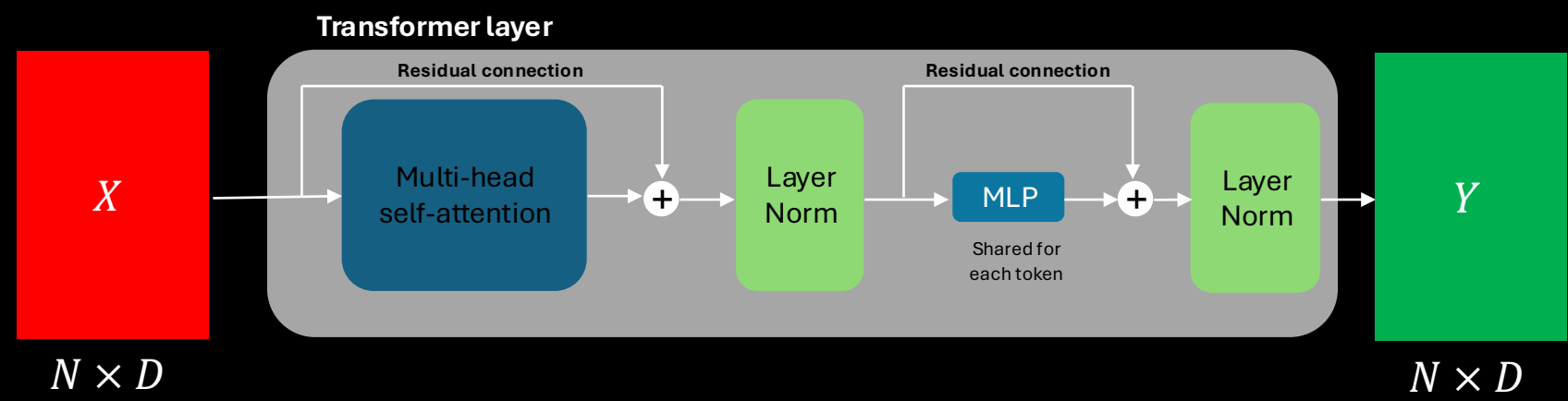
$$Y = \text{Concat}[H_1, H_2, \dots, H_H] W_0$$



Outline of this lecture

- ✓ Motivation
- ✓ Self-attention
- ✓ Scaled self-attention
- ✓ Multi-head self-attention
- Transformer layers
- Positional Encoding
- Transformers for images

Transformer layers



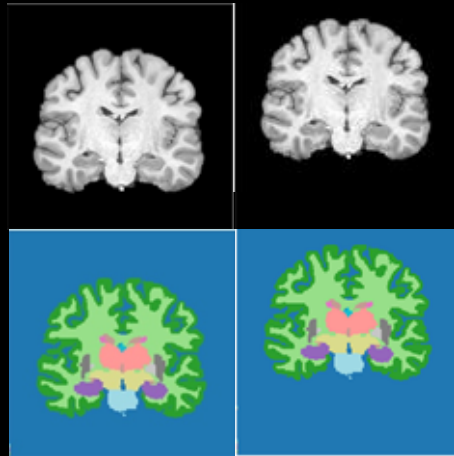
Outline of this lecture

- ✓ Motivation
- ✓ Self-attention
- ✓ Scaled self-attention
- ✓ Multi-head self-attention
- ✓ Transformer layers
 - Positional Encoding
 - Transformers for images

Positional Encoding

- Transformers are equivariant with respect to input permutations!
- Remember equivariance from the CNN lecture – CNNs are translation equivariant

$$f(T(X)) = T(f(X))$$



Positional Encoding

- Transformers are equivariant with respect to input permutations!

$$f(P(X)) = P(f(X))$$

I bought an apple watch

Positional Encoding

- Transformers are equivariant with respect to input permutations!

$$f(P(X)) = P(f(X))$$

I bought an apple watch

watch an apple I bought

Positional Encoding

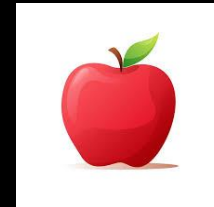
- Transformers are equivariant with respect to input permutations!

$$f(P(X)) = P(f(X))$$

I bought an apple watch



watch an apple I bought



- Both sentences are same for transformers!

Positional Encoding

$$[a_{4,1} \ a_{4,2} \ a_{4,3} \ a_{4,4} \ a_{4,5}] = q_4 \ [k_1^T \ k_2^T \ k_3^T \ k_4^T \ k_5^T]$$

$$[a'_{4,1} \ a'_{4,2} \ a'_{4,3} \ a'_{4,4} \ a'_{4,5}] = \text{Softmax}([a_{4,1} \ a_{4,2} \ a_{4,3} \ a_{4,4} \ a_{4,5}] / \sqrt{D_k})$$

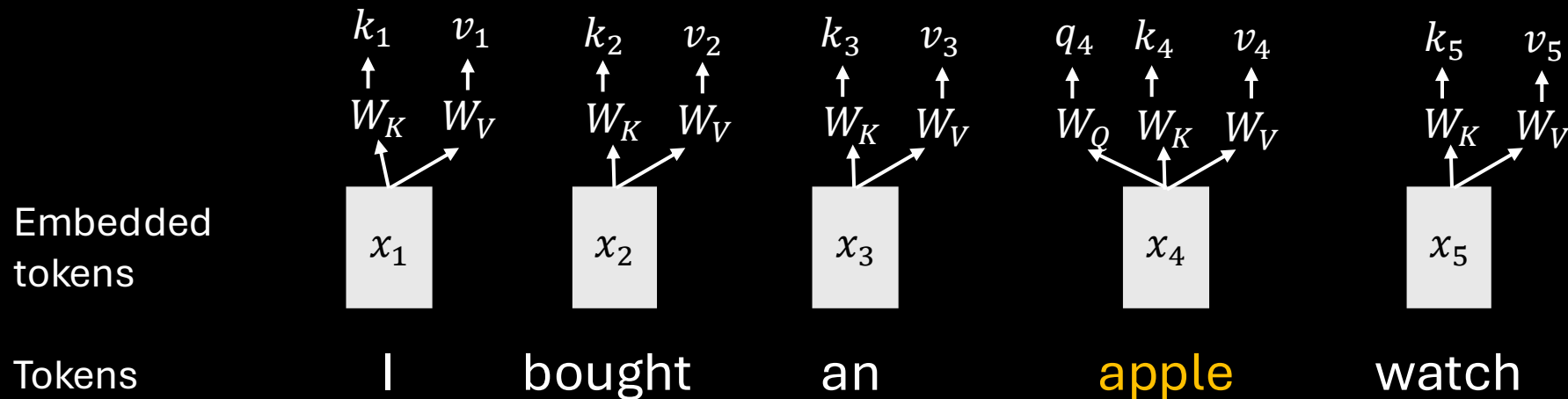
$$\text{Attention}(q_4, K, V) = a'_{4,1}v_1 + a'_{4,2}v_2 + a'_{4,3}v_3 + a'_{4,4}v_4 + a'_{4,5}v_5$$

I bought an **apple** watch

$$\text{Attention}(q_4, K, V) = a'_{4,5}v_5 + a'_{4,3}v_3 + a'_{4,4}v_4 + a'_{4,1}v_1 + a'_{4,2}v_2$$

watch an **apple** I bought

- It is impossible to understand the meaning of the word with only attention
- Solution?
 - Encode token order in the data!



Positional Encoding

- Encode token order in the data!

$$\tilde{x}_i = x_i + r_i \quad r_i, x_i \in \mathbb{R}^{D \times 1}$$

An ideal positional encoding should

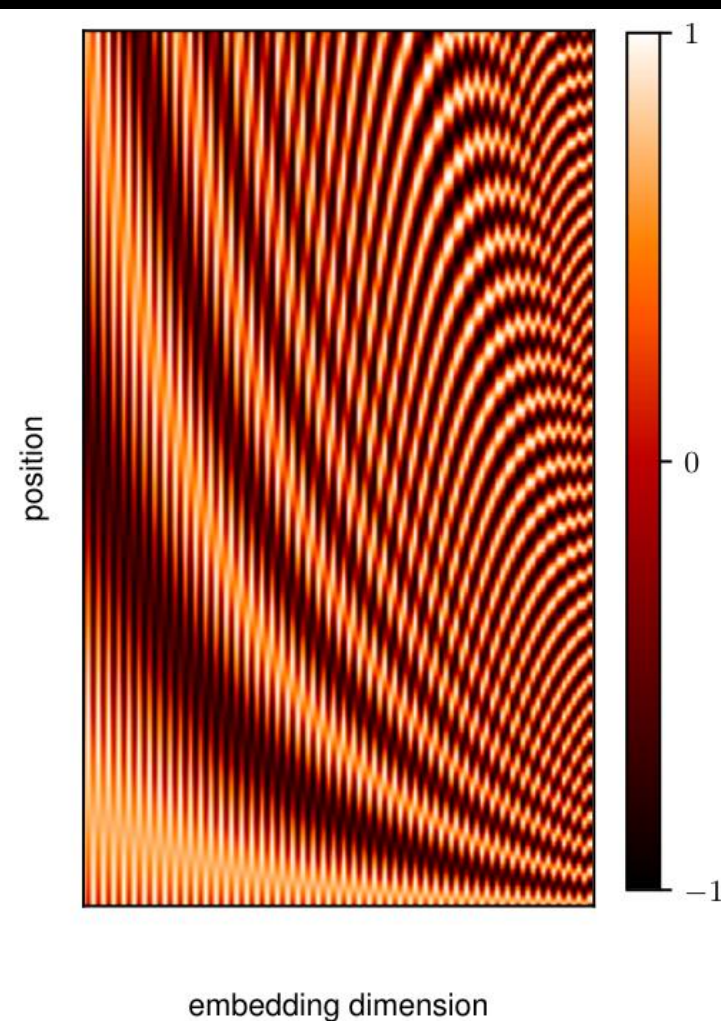
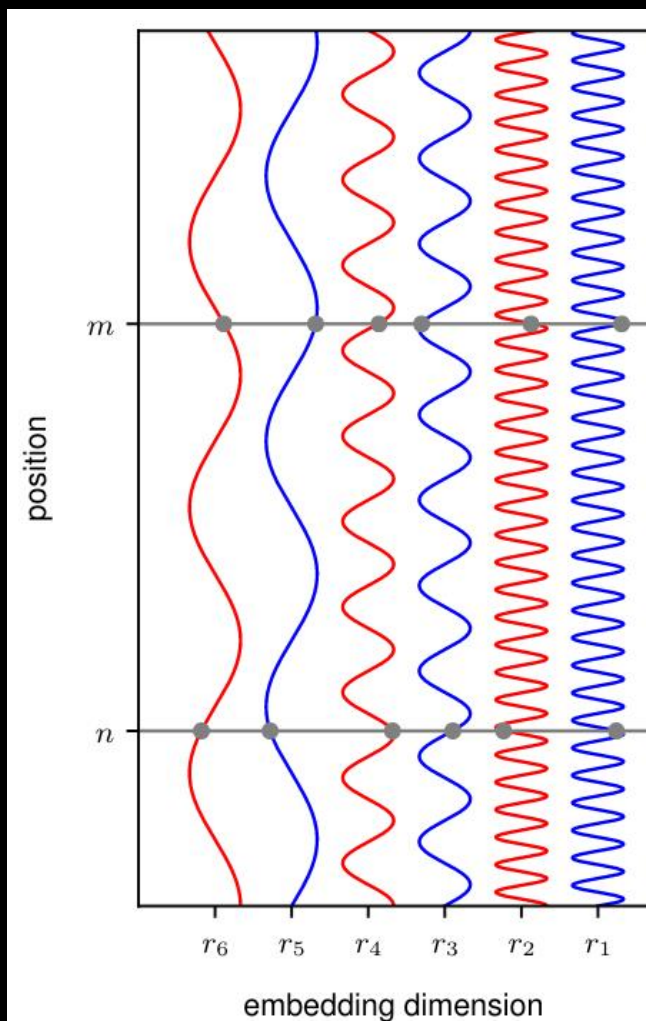
- Provide a unique representation for each position
- Be bounded
- Generalize to longer sequences
- Encode the relative positions of tokens

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$

Positional Encoding

$$r_{ni} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{if } i \text{ is even} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd} \end{cases}$$

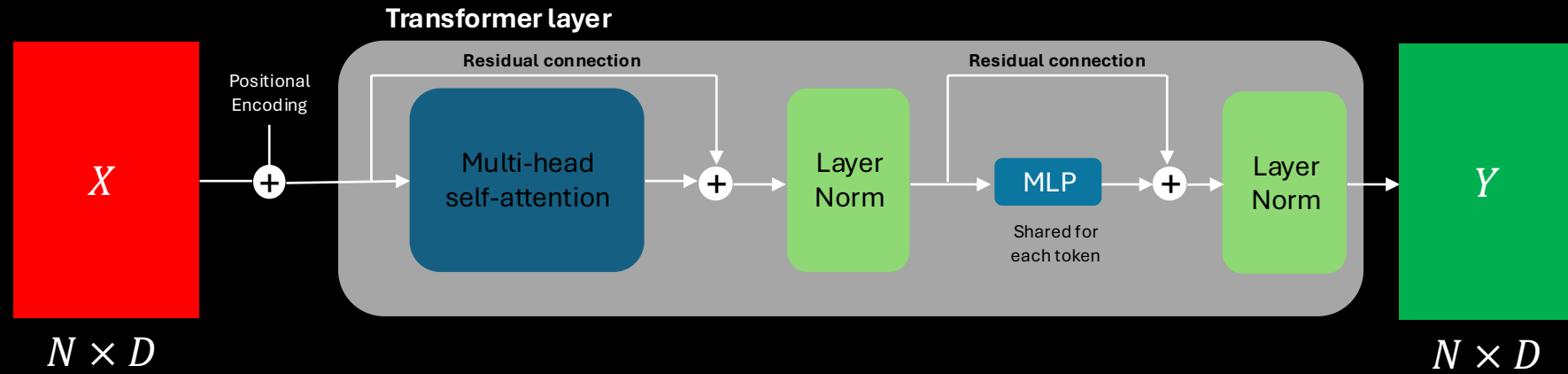
$D = 6$
 $L = 30$
 $N = 200$



$D = 100$
 $L = 30$
 $N = 200$

Positional Encoding

- Transformers with positional encoding



Positional Encoding

- Recent architectures use Rotary Positional Embedding (RoPE)

RoFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING

Jianlin Su
Zhuiyi Technology Co., Ltd.
Shenzhen
bojonesu@wezhuiyi.com

Yu Lu
Zhuiyi Technology Co., Ltd.
Shenzhen
julianlu@wezhuiyi.com

Shengfeng Pan
Zhuiyi Technology Co., Ltd.
Shenzhen
nickpan@wezhuiyi.com

Ahmed Murtadha
Zhuiyi Technology Co., Ltd.
Shenzhen
mengjiayi@wezhuiyi.com

Bo Wen
Zhuiyi Technology Co., Ltd.
Shenzhen
brucewen@wezhuiyi.com

Yunfeng Liu
Zhuiyi Technology Co., Ltd.
Shenzhen
glenliu@wezhuiyi.com

Rotary Position Embedding for Vision Transformer

Byeongho Heo^{ID} Song Park^{ID} Dongyoon Han^{ID} Sangdoo Yun^{ID}

NAVER AI Lab

Outline of this lecture

- ✓ Motivation
- ✓ Self-attention
- ✓ Scaled self-attention
- ✓ Multi-head self-attention
- ✓ Transformer layers
- ✓ Positional Encoding
- Transformers for images

Transformers for images

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

**Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}**

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.¹

Transformers for images

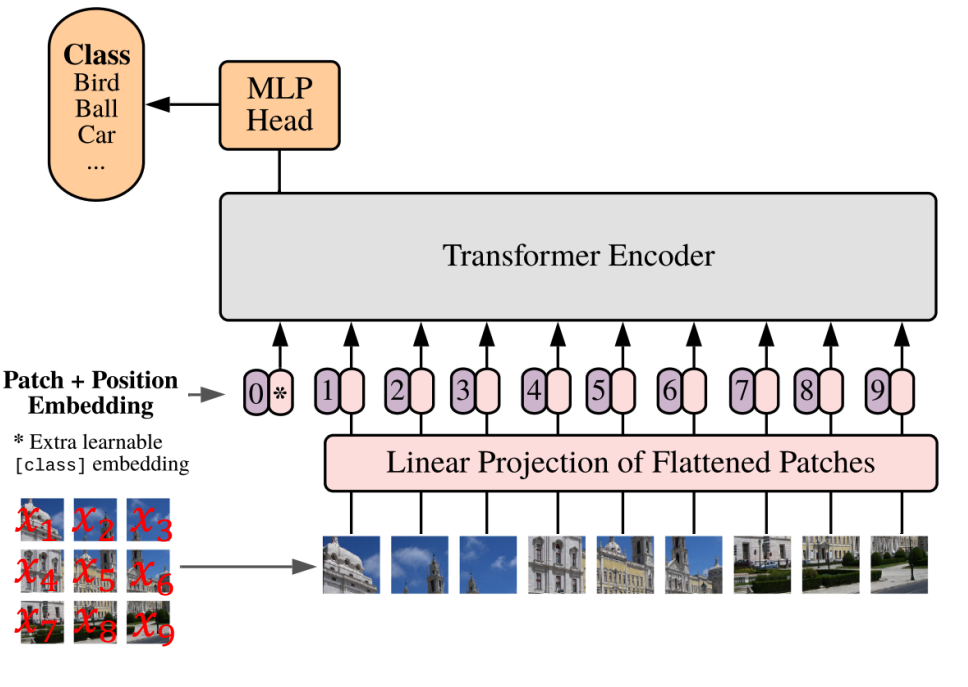
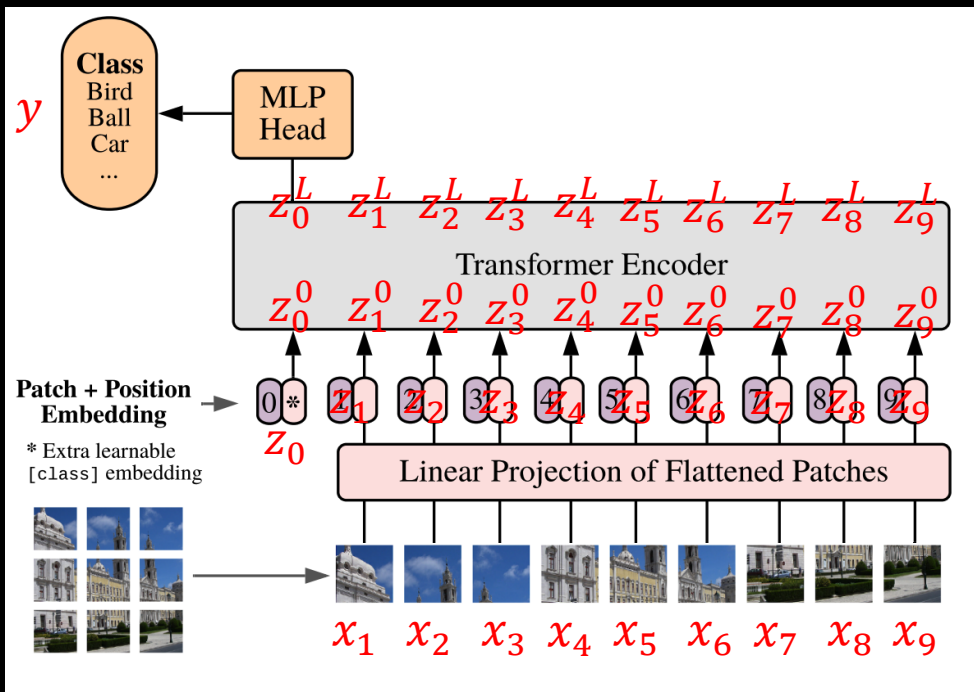


Figure from Attention is All You Need paper by Dosovitsky et al.

Transformers for images



$$y = MLP(z_0^L)$$

$$X_L = TransformerEncoder(X_0)$$

$$X_0 = [z_0^0; z_1^0; \dots; z_9^0]$$

$$z_i^0 = z_i + E_{pos}^i \quad E_{pos}^i \in \mathbb{R}^{1 \times D}$$

$$z_i = flatten(x_i)E \quad E \in \mathbb{R}^{(P^2 C) \times D} \quad z_i \in \mathbb{R}^{1 \times D}$$

$$x_i \in \mathbb{R}^{P \times P \times C} \quad \forall i \in [1, N]$$

$$N = HW/P^2$$

Transformers for images

- CNNs vs Transformers for images
- In CNNs, locality and two-dimensional neighborhood are strong inductive biases for vision tasks.
- In ViT:
 - Only MLP layers are local while self-attention is global.
 - Patching is the only 2-dimensional inductive bias; it has to learn geometrical properties from scratch.
 - Generally, requires more training data than a comparable CNN.

Outline of this lecture

- ✓ Motivation
- ✓ Self-attention
- ✓ Scaled self-attention
- ✓ Multi-head self-attention
- ✓ Transformer layers
- ✓ Positional Encoding
- ✓ Transformers for images

Chapter 12 – Transformers

1 – Deep Learning: Foundations and Concepts by
Christopher Bishop and Hugh Bishop

<https://www.bishopbook.com/>

2 – Understanding Deep Learning by Simon J. Prince

<https://udlbook.github.io/udlbook/>

